FinSight

Educational Stock Market Analysis & Portfolio Management Platform

Project	DESIS Ascend Educare 2025-26
Report Type	Technical Project Report
Team Members	Nency · Pratyaksha · Heena · Garima · Siddhi
Date	29 March 2026
Version	1.0 — Final Submission

Table of Contents

1. **Executive Summary**
2. **Requirements**
3. **Design & Architecture**
    - 3.1 High-Level System Architecture
    - 3.2 Technology Stack
    - 3.3 UML Class Diagram — Core Domain Model
    - 3.4 UML Component Diagram
    - 3.5 UML Sequence Diagram — Student Trade Lifecycle
4. **Working Code Snippets**
    - 4.1 API — Student Trade Submission (Node.js)
    - 4.2 AI — FinBERT Sentiment Classification (Python)
    - 4.3 Frontend — ProtectedRoute with Role Guard (React)
    - 4.4 Quantitative Finance — Diversification & Concentration Scoring
    - 4.5 Data Aggregation and Portfolio Fit
5. **Future Scope**
6. **References**

1. Executive Summary

FinSight is a dual-mode educational fintech platform that bridges the gap between financial literacy and active market participation. The core premise is simple: instead of making trading decisions for users, FinSight uses AI to analyse news, earnings reports, and historical data, empowering users to make informed decisions rather than treating trading as guesswork.

The platform operates in two distinct modes:

- **Student / Supervised Mode** — designed for college students and minors. All trades require explicit guardian approval before execution, with a built-in budget system, notification pipeline, and educational feedback loop.
- **Adult / Pro Mode** — designed for independent traders. Trades execute immediately after the user's own analysis.

Key capabilities at a glance:

- Real-time news aggregation with AI-powered sentiment analysis (FinBERT + OpenAI).
- Automated earnings report detection, extraction, and visualisation.
- Side-by-side stock comparison with risk/return modelling.
- Portfolio diversification scoring using Modern Portfolio Theory (MPT).
- Stock Leaderboard — performance-based ranking within a user's portfolio.
- Guardian-Student co-signing system with approval workflows, budgets, and comments.

The backend is split across two runtimes: Node.js/Express handles identity, sessions, and relational transactions; Python/FastAPI handles all quantitative modelling and AI inference. The frontend is built in React 19 with Vite.

2. Requirements

The following functional requirements were identified and implemented:

#	Requirement	Status
R1	Intelligent News & Social Aggregation — three-tier categorisation (portfolio stocks, selected sectors, general market) integrating official news and social media scraping, with AI-generated analysis per article (OpenAI + Google Gemini).	✓ **Done**
R2	Sentiment Classification — positive / neutral / negative label for each news item, plus reliability score based on historical accuracy.	✓ **Done**
R3	Advanced News Filtering — filter by impact level, time period, sector, and reliability score.	✓ **Done**

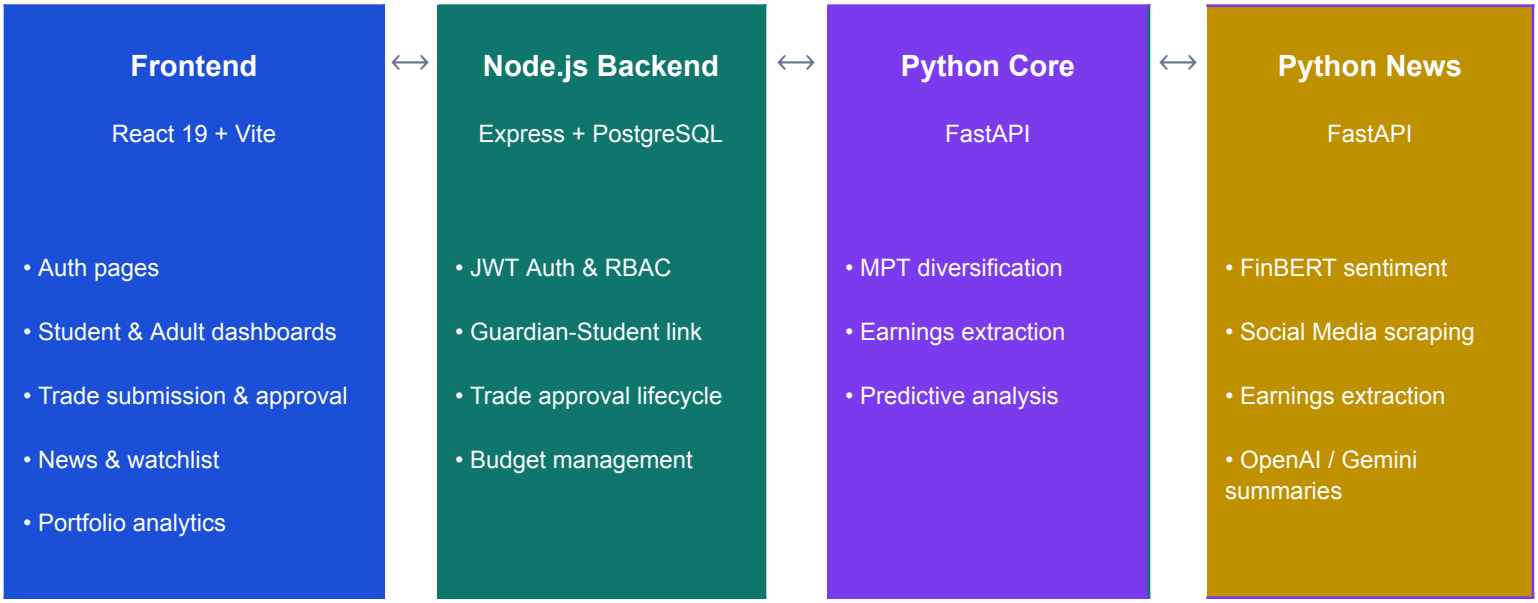
R4	Predictive Analytics & Portfolio Fit — Utilization of historical data and financial metrics to model stock trajectories, leveraging OpenAI to forecast potential performance and evaluate asset compatibility within a user's existing portfolio.	✓ **Done**
R5	Watchlist Module — add stocks with a price target; trigger notifications when target is reached.	✓ **Done**
R6	Stock Comparison Tool — side-by-side comparison of multiple trade candidates with detailed Sharpe Ratio and variance comparisons.	✓ **Done**
R7	Diversification Scoring — 0-100 score based on correlation, sector entropy, and Herfindahl-Hirschman Index concentration.	✓ **Done**
R8	Stock Leaderboard — performance-ranked list of portfolio holdings by ROI and absolute gain.	✓ **Done**
R9	Earnings Analysis — real-time detection of results releases; extract revenue, margins, FCF, EPS; 5-year visualisation.	✓ **Done**
R10	Student Registration with Guardian Invite — tokenised invite link sent to parent on registration.	✓ **Done**
R11	Trade Approval Workflow — student trades remain pending until guardian approves or rejects.	✓ **Done**
R12	Budget Allocation System — guardian sets spending budget; system enforces limits pre-trade.	✓ **Done**
R13	Comment & Feedback System — guardian leaves educational comments on trades visible to the student.	✓ **Done**

R14	Role-Based Access Control — middleware enforces student vs. adult route separation.	✓ **Done**

3. Design & Architecture

3.1 High-Level System Architecture

FinSight uses a microservices-inspired, three-tier architecture:



All services share a PostgreSQL database. Vite proxy routes /api/ to Node (port 3000), and /companies, /financials, etc. to Python (port 8000).*

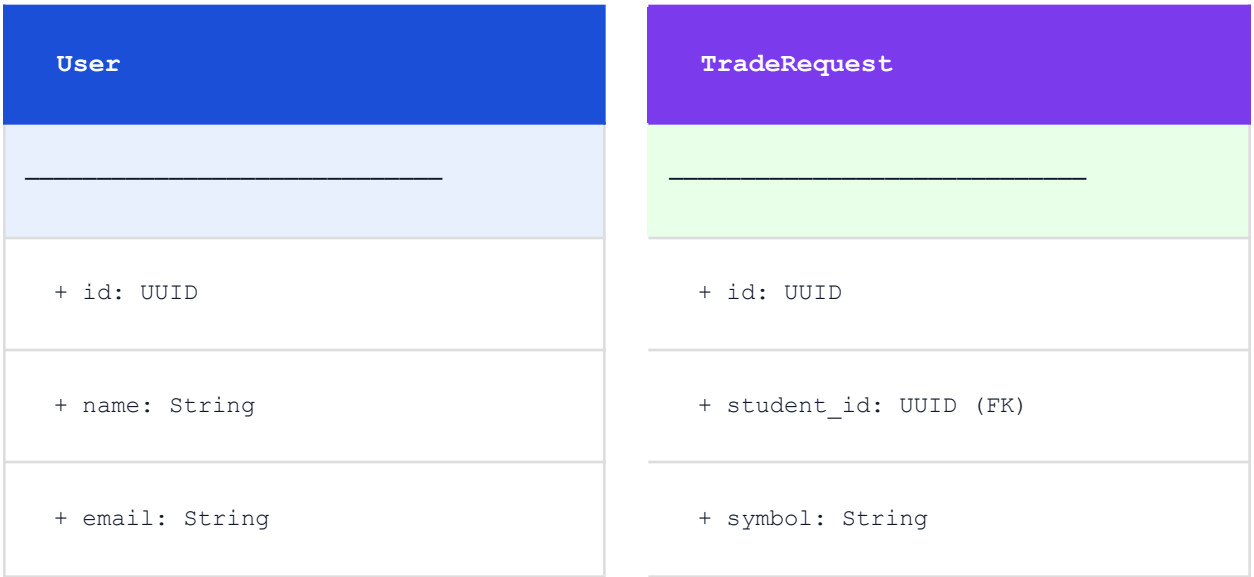
3.2 Technology Stack

Layer	Technology	Purpose
Frontend	**React 19 + Vite**	SPA with role-based routing (ProtectedRoute), React Context for auth state, Recharts for financial visualisations.
Routing / Proxy	**React Router v7 + Vite proxy**	Client-side routing with role guards. Vite proxy transparently forwards API calls to the correct backend.
Node Backend	**Express.js**	REST API for auth, guardian linking, trade lifecycle, and budget management.
Auth / Security	**JWT + bcrypt (cost 12)**	Stateless authentication; passwords hashed with bcrypt; RBAC middleware enforces role separation on every protected route.
Relational DB	**PostgreSQL**	ACID-compliant storage for users, guardian_links, trade_requests. Foreign key constraints maintain referential integrity.
Python Backend	**FastAPI + Uvicorn**	Async REST API for all AI and quantitative operations; Pydantic models for strict input validation.
Quantitative Finance	**NumPy + SciPy + Pandas**	MPT calculations, variance-covariance matrix, Sharpe Ratio, HHI concentration scoring, Monte Carlo simulation.
Market Data	**yfinance**	5-year historical price data for 40 equities; APScheduler cron job refreshes EOD prices daily at 18:30.

Sentiment AI	**FinBERT (HuggingFace)**	Local inference — no API cost. Classifies news headlines into Positive/Neutral/Negative with probability scores.
Generative AI	**OpenAI GPT-5.1 & Google Gemini**	Structured JSON summaries for earnings & portfolio fit; dual-LLM approach for factual outputs and sentiment processing.
News Ingestion	**httpx (async)**	Concurrent fetching from NewsAPI, Yahoo Finance RSS, Google News RSS, and targeted social media scraping pipelines; SHA-256 deduplication.
ORM / DB Client	**SQLAlchemy + node-postgres**	Python side uses SQLAlchemy for upserts; Node side uses pg (node-postgres) raw queries for performance.

3.3 UML Class Diagram — Core Domain Model

The following diagram shows the primary database-backed entities and their relationships:



```
+ password_hash: String
+ role: Enum(student|adult)
+ budget: Decimal?
+ budget_used: Decimal
+ created_at: Timestamp
____________________________
+ findByEmail(email): User
+ findById(id): User
+ setBudget(id, amt): User
+ updateBudgetUsed(id,a): User
```

```
+ qty: Integer
+ action: Enum(buy|sell)
+ estimated_price: Decimal
+ reasoning: Text?
+ status: Enum(pending|...)
+ parent_comment: Text?
+ resolved_at: Timestamp?
____________________________
+ createTradeRequest(): TR
+ resolve(id, status): TR
```

```
GuardianLink
____________________________
+ id: UUID
+ student_id: UUID (FK)
```

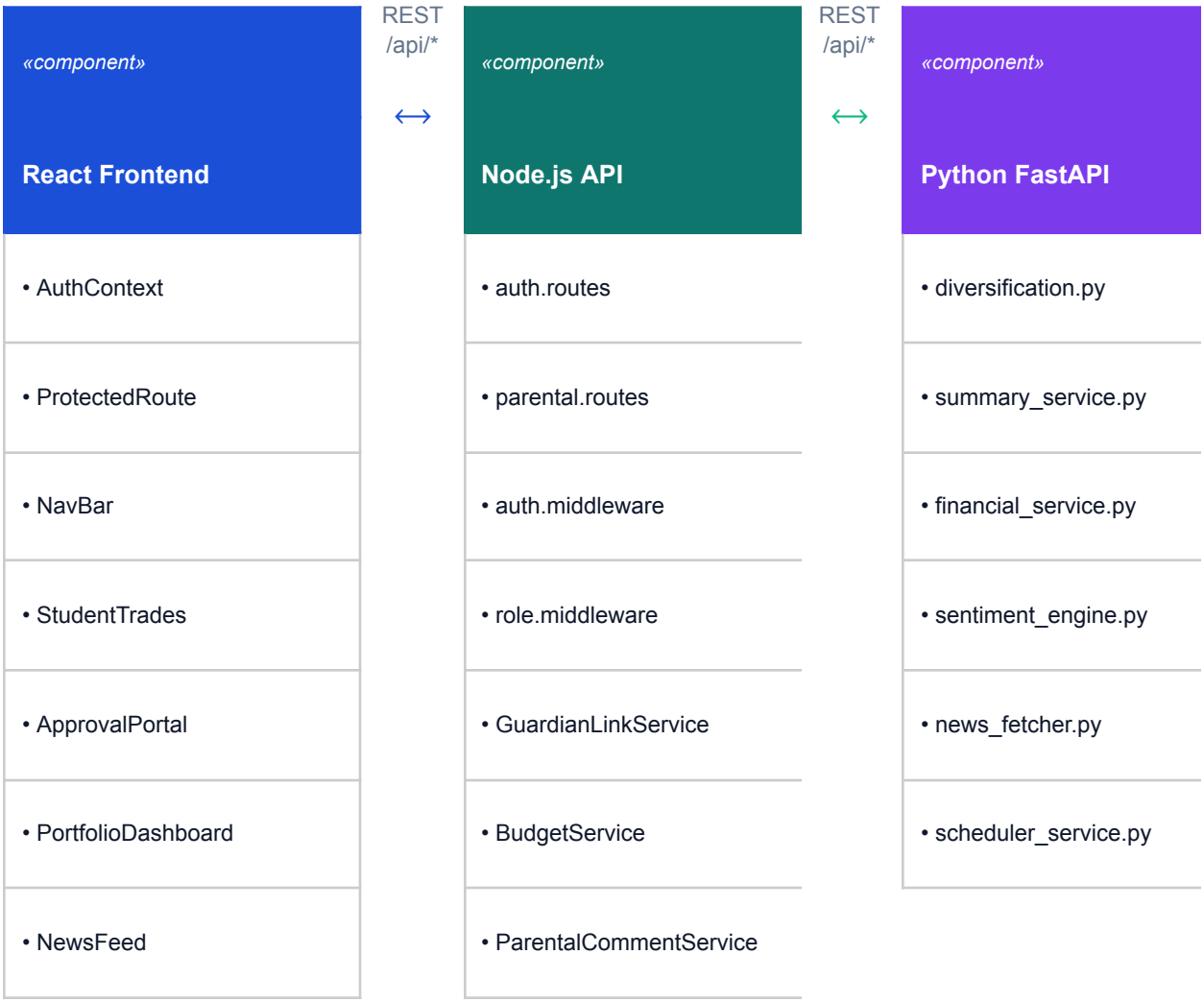
```
HistoricalReport  (Python DB)
____________________________
+ id: Integer
+ ticker: String
```

+ guardian_id: UUID (FK)	+ fiscal_year: Integer
+ status: Enum(pending|active)	+ total_revenue: Float
+ invite_token: String	+ gross_margin: Float
+ linked_at: Timestamp?	+ net_income: Float
————————————————	+ basic_eps: Float
+ createLink(studentId): Link	+ free_cash_flow: Float
+ activateLink(token, gId): Link	————————————————
+ getStudentsForGuardian(gId)	Populated by: financial_service.py
	Cron: daily @ 18:30

Relationships: User 1 ──< GuardianLink (student side) | User 1 ──< GuardianLink (guardian side) | User (student) 1 ──< TradeRequest

3.4 UML Component Diagram

The diagram below shows the major software components and their communication interfaces:



All three tiers share a single PostgreSQL instance. The Python backend additionally integrates with HuggingFace (local FinBERT) and OpenAI (cloud GPT-5.1) for AI inference.

3.5 UML Sequence Diagram — Student Trade Lifecycle

This diagram traces the complete flow from a student submitting a trade to a guardian resolving it:

Actor / Component	Dir	Target	Message / Action
Student (UI)	→	**Node API**	POST /api/parental/trades { symbol, qty, price, reasoning }
Node API	→	**BudgetService**	checkSufficiency(studentId, totalCost)
Node API	→	**GuardianModel**	getLink(studentId) — verify active guardian exists
Node API	→	**DB (PostgreSQL)**	INSERT INTO trade_requests (status='pending')
Node API	←	**DB (PostgreSQL)**	Return new trade row
Student (UI)	←	**Node API**	201 Created — trade visible in history as ⏳ Pending
Guardian (UI)	→	**Node API**	GET /api/parental/trades/pending
Guardian (UI)	→	**Node API**	PUT /api/parental/trades/:id/approve { comment }
Node API	→	**GuardianLinkService**	approveTrade(guardianId, tradeId, comment)
GuardianLinkService	→	**DB (PostgreSQL)**	UPDATE trade_requests SET status='approved', parent_comment=…
GuardianLinkService	→	**UserModel**	updateBudgetUsed(studentId, qty × price)
Guardian (UI)	←	**Node API**	200 OK — trade shown as ✓ Approved

Student (UI)	→	**Node API**	GET /api/parental/trades/mine — sees resolved trade + comment

4. Working Code Snippets

4.1 API — Student Trade Submission (Node.js)

The submitTradeRequest service enforces two guards before inserting the trade: budget sufficiency and an active guardian link.

guardian-link.service.js — submitTradeRequest()

`backend-node/services/guardian-link.service.js`

```
async submitTradeRequest(student, tradeData) {
  const { symbol, qty, action, estimatedPrice, reasoning } = tradeData;
  const totalCost = qty * estimatedPrice;

  const isAdult = student.role === 'adult';

  if (!isAdult) {
    if (action === 'buy') {
      if (student.budget === null || student.budget === undefined) {
        throw new Error('No budget allocated yet. Ask your guardian to set a budget before trading.');
      }
      const remaining = student.budget - (student.budget_used || 0);
      if (totalCost > remaining) {
        throw new Error(`Insufficient budget. Available: $${remaining.toFixed(2)}`);
      }
    }
    // Check student has an active guardian link
    const link = await GuardianModel.getLink(student.id);
    if (!link) throw new Error('No active guardian linked. Ask your parent to accept your invite.');
  }

  const trade = await GuardianModel.createTradeRequest({
          studentId: student.id, symbol, qty, action, estimatedPrice, reasoning,
          status: isAdult ? 'approved' : 'pending',
        });
  if (isAdult) {
    await updateHoldings(student.id, symbol, qty, action, estimatedPrice);
  }
  return trade;
},
```

4.2 AI — FinBERT Sentiment Classification (Python)

The sentiment engine runs FinBERT locally, then computes a weighted impact score from sentiment strength, keyword urgency, source credibility, and ticker relevance.

sentiment_engine.py — Weighted Impact Score

`news-service/app/processing/sentiment_engine.py`

```python
async def compute_impact_score(sentiment_score: float, title: str, is_primary: bool, source_credibility: float = 0.5):
    # Expanded High-impact keywords with weights
    CRITICAL_KEYWORDS = ["bankruptcy", "fraud", "ceo", "acquisition", "merger", "earnings", "guidance"]
    URGENT_KEYWORDS = ["fda", "lawsuit", "layoff", "dividend", "rating", "sec"]

    title_lower = title.lower()

    keyword_score = 0.0
    if any(kw in title_lower for kw in CRITICAL_KEYWORDS):
        keyword_score = 0.4
    elif any(kw in title_lower for kw in URGENT_KEYWORDS):
        keyword_score = 0.2

    sentiment_strength = abs(sentiment_score - 0.5) * 2
    primary_weight = 0.2 if is_primary else 0.05

    # Advanced Weighted Formula
    # 30% Sentiment, 30% Keywords, 20% Source Trust, 20% Ticker Relevance
    raw_score = (
        (0.3 * sentiment_strength) +
        (0.3 * keyword_score) +
        (0.2 * source_credibility) +
        (0.2 * primary_weight)
    )
    impact_score = min(1.0, raw_score * 2.0) # Normalized
    if impact_score >= 0.75:
        level = "HIGH"
    elif impact_score >= 0.4:
        level = "MEDIUM"
    else:
        level = "LOW"
    return impact_score, level
```

4.3 Frontend — ProtectedRoute with Role Guard (React)

Every sensitive page is wrapped in ProtectedRoute, which reads the auth context and redirects unauthorized users before rendering any content.

ProtectedRoute.jsx — Role-Based Route Guard

`frontend/src/components/layout/ProtectedRoute.jsx`

```
import { Navigate } from 'react-router-dom';
import { useAuth } from '../../hooks/useAuth';

export default function ProtectedRoute({ children,
requiredRole }) {
  const { user, loading } = useAuth();

  if (loading) {
    return (
      <div style={{ display: 'flex', alignItems: 'center',
      justifyContent: 'center', minHeight: '100vh' }}>
        <p style={{ color: 'var(--text-muted)', fontSize: 14 }}
        >Loading…</p>
      </div>
    );
  }

  if (!user) return <Navigate to="/login" replace />;

  if (requiredRole && user.role !== requiredRole) {
    return <Navigate to={user.role === 'student' ? '/student/
    dashboard' : '/adult/dashboard'} replace />;
  }

  return children;
}
```

4.4 Quantitative Finance — Diversification & Concentration Scoring

Every sensitive page is wrapped in ProtectedRoute, which reads the auth context and redirects unauthorized users before rendering any content.

finance_service.py— Diversification Scoring

backend-python/app/services/finance_service.py

```python
def compute_correlation_matrix(returns_dict: dict):
    tickers = list(returns_dict.keys())
    matrix = {}
    for t in tickers:
        matrix[t] = {}
        for t2 in tickers:
            if t == t2:
                matrix[t][t2] = 1.0
            else:
                r, _ = pearsonr(returns_dict[t], returns_dict[t2])
                matrix[t][t2] = round(float(r), 4)
    return matrix, tickers


def average_pairwise_correlation(returns_dict: dict) -> float:
    tickers = list(returns_dict.keys())
    n = len(tickers)
    if n < 2:
        return 0.0
    total, count = 0.0, 0
    for i in range(n):
        for j in range(i + 1, n):
            r, _ = pearsonr(returns_dict[tickers[i]], returns_dict[tickers[j]])
            total += r
            count += 1
    return total / count


def sector_diversity_bonus(holdings: list) -> float:
    sector_counts = {}
    for h in holdings:
        s = h["sector"]
        sector_counts[s] = sector_counts.get(s, 0) + 1
    total = sum(sector_counts.values())
    probs = [c / total for c in sector_counts.values()]
    n = len(probs)
    if n == 1:
        return 0.5
    entropy = -sum(p * np.log(p) for p in probs if p > 0)
    max_entropy = np.log(n)
    normalized = entropy / max_entropy
    return 0.7 + normalized * 0.5
```

```python
def concentration_penalty(holdings: list) -> float:
    total_value = sum(h["value"] for h in holdings)
    weights = [h["value"] / total_value for h in holdings]
    hhi = sum(w ** 2 for w in weights)
    n = len(weights)
    min_hhi = 1 / n
    normalized_hhi = (hhi - min_hhi) / (1 - min_hhi + 1e-9)
    return normalized_hhi


def compute_diversification_score(holdings: list, returns_dict: dict) -> float:
    avg_corr = average_pairwise_correlation(returns_dict)
    corr_score = (1 - avg_corr) * 100

    bonus = sector_diversity_bonus(holdings)
    sector_score = ((bonus - 0.7) / 0.5) * 100

    penalty = concentration_penalty(holdings)
    conc_score = (1 - penalty) * 100

    final = 0.5 * corr_score + 0.3 * sector_score + 0.2 * conc_score
    return round(max(0, min(100, final)), 1)
```

4.5 Data Aggregation and Portfolio Fit

This module gathers the user's portfolio and historical stock data, calculates the risk-adjusted portfolio fit using Sharpe Ratios, and constructs the final payload for the AI.

summary_service.py— Portfolio Fit

`backend-python/app/services/summary_service.py`

```python
# 4. Calculate Expected Returns and Covariance (MPT Math)
mean_returns = returns.mean() * 252
cov_matrix = returns.cov() * 252

def get_portfolio_metrics(weights_array, ticker_list):
    port_return = np.dot(weights_array, mean_returns[ticker_list].values)
    cov_vals = cov_matrix.loc[ticker_list, ticker_list].values

    port_volatility = np.sqrt(max(0, np.dot(weights_array.T, np.dot(cov_vals, weights_array))))
    sharpe_ratio = (port_return - risk_free_rate) / port_volatility if port_volatility > 0 else 0
    return sharpe_ratio

# 5. Evaluate Current Portfolio
current_weights = np.array([current_weights_dict[t] for t in tickers])
current_sharpe = get_portfolio_metrics(current_weights, tickers)

# 6. Evaluate Proposed Portfolio (10% allocation)
proposed_allocation = 0.10
proposed_tickers = tickers.copy()

if candidate_upper not in proposed_tickers:
    proposed_tickers.append(candidate_upper)
    new_weights = [w * (1 - proposed_allocation) for w in current_weights]
    new_weights.append(proposed_allocation)
else:
    idx = proposed_tickers.index(candidate_upper)
    new_weights = [w * (1 - proposed_allocation) for w in current_weights]
    new_weights[idx] += proposed_allocation

proposed_weights = np.array(new_weights)
proposed_sharpe = get_portfolio_metrics(proposed_weights, proposed_tickers)

# 7. Map to Score
sharpe_delta = proposed_sharpe - current_sharpe
raw_score = 50 + (sharpe_delta * 100)
fit_score = max(0, min(100, int(round(raw_score))))
```

5. Future Scope

FinSight is designed with extensibility in mind. The following enhancements are planned for future iterations:

Real-Time WebSocket Price Feeds

The current architecture uses REST polling for price updates. A WebSocket layer (e.g., Socket.io on Node.js) would push live ticker data and guardian approval notifications to the frontend without polling overhead.

Advanced Risk Metrics

The diversification engine currently computes Sharpe Ratio and HHI. Future versions will incorporate Beta (market volatility correlation), Maximum Drawdown analysis, and the Sortino Ratio (downside-only risk penalty) for deeper portfolio insights.

Automated Rebalancing Suggestions

Building on the existing diversification scorer, the AI engine would suggest specific buy/sell actions to bring a portfolio closer to a user-defined target allocation, completing the advisory loop.

Scaling Predictive Models

The current system utilizes historical data to fuel baseline predictive analytics. Future iterations will refine these models by integrating advanced machine learning techniques (such as LSTMs or Prophet) over significantly larger historical datasets to improve trajectory accuracy.

Email & Push Notifications

The notification templates (approval-request.html, price-alert.html, earnings-alert.html) are already scaffolded. Full integration with an email provider (e.g., SendGrid) and PWA push notifications is a near-term roadmap item.

Expanding Social Sentiment Indexing

The existing news ingestion pipeline successfully scrapes targeted social media. We plan to scale this architecture to ingest high-velocity data from additional social platforms, creating a highly responsive "retail trader sentiment" index to run alongside traditional financial news.

Mobile Application

Packaging the React frontend as a Progressive Web App (PWA) or wrapping it with React Native would extend FinSight to iOS and Android, especially valuable for the student user segment.

Expanded Asset Universe

The current data pipeline tracks 40 equities. Future releases would expand to ETFs, mutual funds, and fixed-income instruments, enabling more comprehensive diversification analysis for adult users.

6. References

The following libraries, models, APIs, and publications were used or consulted during the development of FinSight:

Libraries & Frameworks

- React 19 — https://react.dev
- React Router v7 — https://reactrouter.com
- Vite — https://vitejs.dev
- Recharts — https://recharts.org
- Express.js — https://expressjs.com
- node-postgres (pg) — https://node-postgres.com
- jsonwebtoken — https://github.com/auth0/node-jsonwebtoken
- bcrypt — https://github.com/kelektiv/node.bcrypt.js
- FastAPI — https://fastapi.tiangolo.com
- SQLAlchemy — https://sqlalchemy.org
- Pandas — https://pandas.pydata.org
- NumPy / SciPy — https://numpy.org / https://scipy.org
- APScheduler — https://apscheduler.readthedocs.io
- httpx — https://www.python-httpx.org

AI Models & APIs

- ProsusAI/FinBERT (HuggingFace Transformers) — https://huggingface.co/ProsusAI/finbert
- OpenAI GPT-5.1 API — https://platform.openai.com/docs
- Google Gemini API — https://ai.google.dev/docs
- yfinance — https://github.com/ranaroussi/yfinance
- NewsAPI — https://newsapi.org

Academic & Conceptual References

- Markowitz, H. M. (1952). Portfolio Selection. Journal of Finance, 7(1), 77–91. — Foundation for MPT and diversification scoring.
- Sharpe, W. F. (1966). Mutual Fund Performance. Journal of Business, 39(1), 119–138. — Sharpe Ratio formulation.
- Herfindahl-Hirschman Index — US Department of Justice, Merger Guidelines. — Concentration scoring basis.
- Araci, D. (2019). FinBERT: Financial Sentiment Analysis with Pre-trained Language Models. arXiv:1908.10063.

Development Tools

- Visual Studio Code — https://code.visualstudio.com
- PostgreSQL 16 — https://www.postgresql.org
- Postman — API testing — https://www.postman.com
- Git & GitHub — Version control and team collaboration